

Royaume du Maroc
Ministère de l'Éducation Nationale, du Préscolaire et des Sports
Académie Régionale de l'Éducation et de la Formation Drâa-Taflalet
Direction Provinciale de Ouarzazate



Rapport de Projet de Fin d'Études

Deuxième année BTS
Développement des Applications Informatiques

**Conception et réalisation d'une application mobile de
gestion des commandes d'un restaurant**

Mon PF App

Projet	Mon PF App
Réalisé par	HAJJI Yassmine
Établissement	Lycée Qualifiant Technique Ibn Al-Haitam
Formation	BTS Développement des Applications Informatiques
Encadrement	Encadrant pédagogique : à compléter
Année de formation	2024 – 2026

Remerciements

Je tiens à remercier l'équipe pédagogique du BTS Développement des Applications Informatiques du Lycée Qualifiant Technique Ibn Al-Haitam pour l'encadrement, les connaissances techniques et la méthodologie acquises durant ma formation.

Je remercie également toutes les personnes qui ont participé à la définition du besoin, aux tests, aux retours d'utilisation et à l'amélioration progressive de l'application **Mon PF App**.

Ce projet m'a permis de rapprocher les acquis de programmation, base de données, analyse UML, sécurité, tests et documentation d'un besoin applicatif concret : la gestion des commandes d'un restaurant.

Résumé

Ce rapport présente **Mon PF App**, une application mobile de gestion des commandes et des livraisons d'un restaurant. Le projet répond à la problématique de la gestion manuelle : erreurs de commande, retards de préparation, manque de suivi et communication insuffisante entre client, restaurant et livreur.

La solution s'appuie sur **Flutter** pour l'application mobile et **Laravel** pour l'API backend. Le système permet au client de consulter le menu, gérer le panier, passer une commande et suivre son état. Il propose aussi des espaces dédiés à l'administrateur, au serveur et au livreur.

La version livrée est une version **démo-prod** : elle dispose d'une interface aboutie, d'un backend local fonctionnel, d'une base SQLite seedée, de tests automatiques et d'une documentation permettant d'installer, tester, debugger et présenter le projet.

Table des matières

Remerciements	i
Résumé	ii
1 Introduction générale	1
2 Contexte et problématique	2
3 Objectifs du projet	3
4 Cahier des charges	4
4.1 Besoins fonctionnels	4
4.2 Besoins non fonctionnels	4
4.3 Couverture actuelle	4
5 Analyse des acteurs	6
6 Analyse fonctionnelle	7
7 Conception des données	8
8 Architecture technique	10
9 Réalisation backend Laravel	12
9.1 Routes principales	12
9.2 Composants backend	12
10 Réalisation Flutter	13
11 Sécurité	15
12 Algorithmes et règles métier	16
12.1 Gestion du panier	16
12.2 Calcul du total	16
12.3 Règles de commande	16
13 Interface et expérience utilisateur	17
14 Tests et validation	18

15 Installation et mise en service locale	19
15.1 Lancer le backend	19
15.2 Lancer Flutter avec l'API locale	19
16 Android et Windows	20
17 Difficultés rencontrées	21
18 Validation du cahier de charge	22
19 Perspectives	23
20 Conclusion	24
A Comptes de démonstration	25
B Fichiers importants	26

1 Introduction générale

Dans le cadre de la formation BTS Développement des Applications Informatiques, le projet de fin d'études doit démontrer la capacité à analyser un besoin, concevoir une solution, réaliser une application, tester son fonctionnement et documenter le travail effectué.

Mon PF App est une application de gestion des commandes d'un restaurant. Elle montre un parcours client complet et des espaces opérationnels pour l'administration et la livraison.

Le choix du sujet est pertinent pour un PFE car il mobilise plusieurs compétences :

- analyse des acteurs et des besoins ;
- modélisation des données ;
- développement mobile Flutter ;
- création d'une API REST Laravel ;
- sécurité et gestion des rôles ;
- tests et génération d'un APK Android ;
- documentation technique et académique.

TABLE 1.1 – Fiche signalétique du projet

Élément	Description
Sujet	Application mobile de gestion des commandes d'un restaurant
Projet	Mon PF App
Stagiaire	HAJJI Yassmine
Formation	BTS Développement des Applications Informatiques
Cibles	Android et Windows

2 Contexte et problématique

Les restaurants gèrent de plus en plus de commandes à emporter et en livraison. Lorsque les commandes sont prises manuellement, les erreurs de saisie, les oublis et les retards deviennent fréquents.

Le client souhaite connaître l'état de sa commande. Le restaurant doit organiser les préparations. Le livreur doit savoir quelles commandes sont disponibles et lesquelles lui sont affectées.

La problématique du projet est donc la suivante :

Comment digitaliser le processus de commande afin de réduire les erreurs humaines, accélérer le traitement et améliorer la communication entre les acteurs ?

TABLE 2.1 – Problèmes identifiés et réponses du projet

Problème	Conséquence	Réponse Mon PF App
Erreur de commande	Plat incorrect ou quantité fausse	Panier numérique et validation explicite
Retard de préparation	Client insatisfait	Suivi par statut
Mauvaise communication	Livreur ou restaurant mal informé	Espaces dédiés par acteur
Menu difficile à maintenir	Prix ou disponibilité non à jour	Catalogue API et endpoints de gestion menu

3 Objectifs du projet

L'objectif général est de réaliser une application mobile de gestion des commandes d'un restaurant. Elle doit organiser le processus depuis le choix des plats jusqu'à la livraison.

Les objectifs spécifiques sont :

- permettre aux clients de consulter le menu ;
- ajouter des plats au panier ;
- calculer automatiquement le montant total ;
- passer une commande ;
- suivre l'état de la commande ;
- permettre au livreur de consulter et accepter les livraisons ;
- permettre à l'administrateur de consulter les statistiques et les commandes ;
- structurer le projet de manière professionnelle pour faciliter la maintenance.

Sur le plan technique, le projet doit être testable et démontrable. Le repository a donc été organisé avec un frontend Flutter, un backend Laravel, des documents de spécification, des guides et des scripts.

4 Cahier des charges

Le cahier de charge fourni indique les besoins fonctionnels côté client, restaurant et livreur. Il indique aussi les besoins non fonctionnels : sécurité, simplicité, rapidité, fiabilité et maintenance.

4.1 Besoins fonctionnels

TABLE 4.1 – Besoins fonctionnels par acteur

Acteur	Fonctionnalités attendues
Client	Créer un compte, se connecter, consulter le menu, ajouter des plats au panier, passer une commande, suivre l'état de la commande.
Administrateur / restaurant	Gérer les utilisateurs, gérer le menu, recevoir les commandes, mettre à jour l'état des commandes, affecter un livreur, consulter les statistiques.
Serveur	Traiter les commandes sur place et participer au flux opérationnel du restaurant.
Livreur	Se connecter, consulter les commandes à livrer, accepter une livraison, mettre à jour l'état de livraison, consulter l'historique.

4.2 Besoins non fonctionnels

- sécurité des données et authentification ;
- interface mobile simple et intuitive ;
- rapidité de traitement des commandes ;
- fiabilité et synchronisation des états ;
- facilité de maintenance.

4.3 Couverture actuelle

La version actuelle couvre le coeur du cahier de charge pour une démonstration PFE :

- le client peut se connecter, consulter le menu, gérer son panier, passer une commande et suivre son statut ;

- l'administrateur dispose d'une vision statistiques et commandes ;
- le livreur peut consulter la file et accepter une livraison ;
- le backend Laravel fournit les données persistantes locales.

5 Analyse des acteurs

Le système est conçu autour de quatre acteurs principaux.

TABLE 5.1 – Acteurs du système

Acteur	Responsabilités
Client	Consulter le menu, gérer le panier, commander et suivre la livraison.
Administrateur	Superviser les commandes, consulter les statistiques, gérer le menu et les affectations.
Serveur	Intervenir dans le traitement opérationnel des commandes du restaurant.
Livreur	Voir les commandes à livrer, accepter une livraison et mettre à jour son état.

Cette séparation permet de contrôler les accès. Un client ne doit pas pouvoir se créer un compte administrateur ou livreur. Les rôles sensibles sont donc imposés côté serveur.

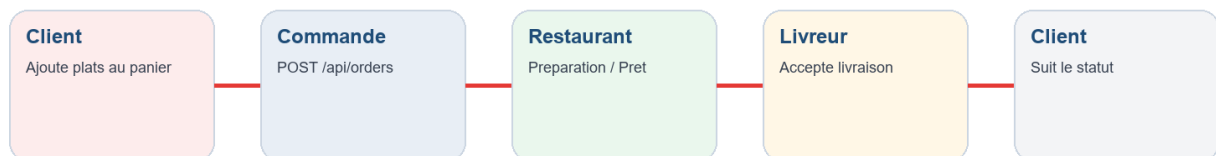
6 Analyse fonctionnelle

L'analyse fonctionnelle décompose le système en cas d'utilisation. Chaque cas d'utilisation correspond à une action visible dans l'application ou dans l'API.

Le parcours principal est le passage de commande :

1. le client se connecte ;
2. il consulte le menu ;
3. il ajoute des plats au panier ;
4. il valide le panier ;
5. l'API crée une commande ;
6. le restaurant et le livreur peuvent traiter cette commande ;
7. le client suit son état.

Cycle de commande



Regle: une commande ne peut pas être créée sans plats. Les statuts sont contrôlés par l'API.

FIGURE 6.1 – Cycle de commande dans Mon PF App

7 Conception des données

Le modèle de données s'organise autour des utilisateurs, catégories, plats, commandes et lignes de commande. Cette structure évite de stocker une commande comme simple texte et permet de garder l'historique des prix et quantités.

Modele logique simplifie

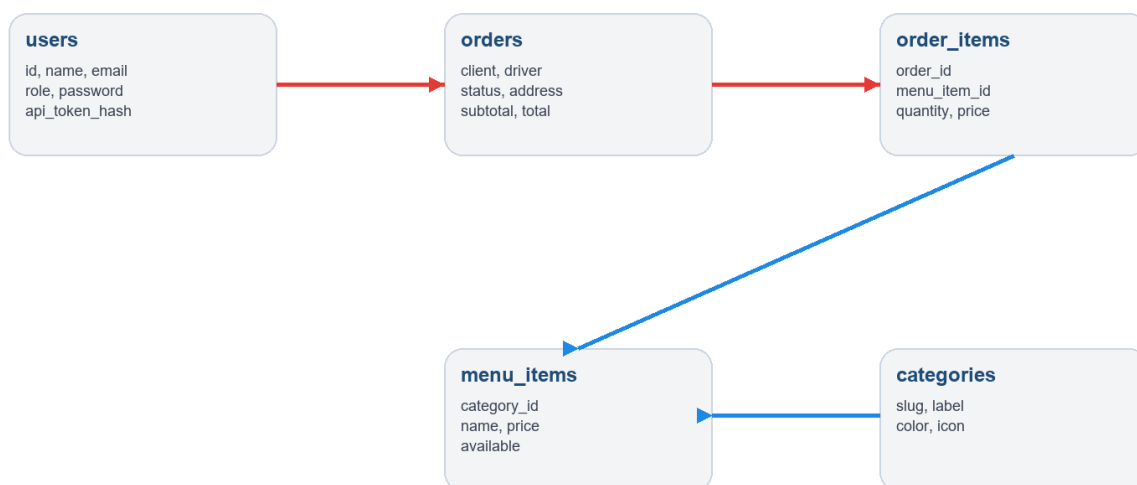


FIGURE 7.1 – Modèle logique simplifié du backend

TABLE 7.1 – Entités principales

Entité	Rôle
User	Stocke les informations utilisateur, le rôle et le hash du token.
Category	Organise les plats par familles.
MenuItem	Représente un plat avec son prix, sa catégorie, son image et sa disponibilité.
Order	Représente une commande client avec statut, adresse, total et livreur affecté.

OrderItem	Stocke les lignes de commande afin de conserver le prix et la quantité commandés.
-----------	---

8 Architecture technique

L'architecture sépare clairement le frontend et le backend. Flutter gère l'interface, l'état utilisateur et le panier. Laravel gère les validations, les routes REST et la persistance.

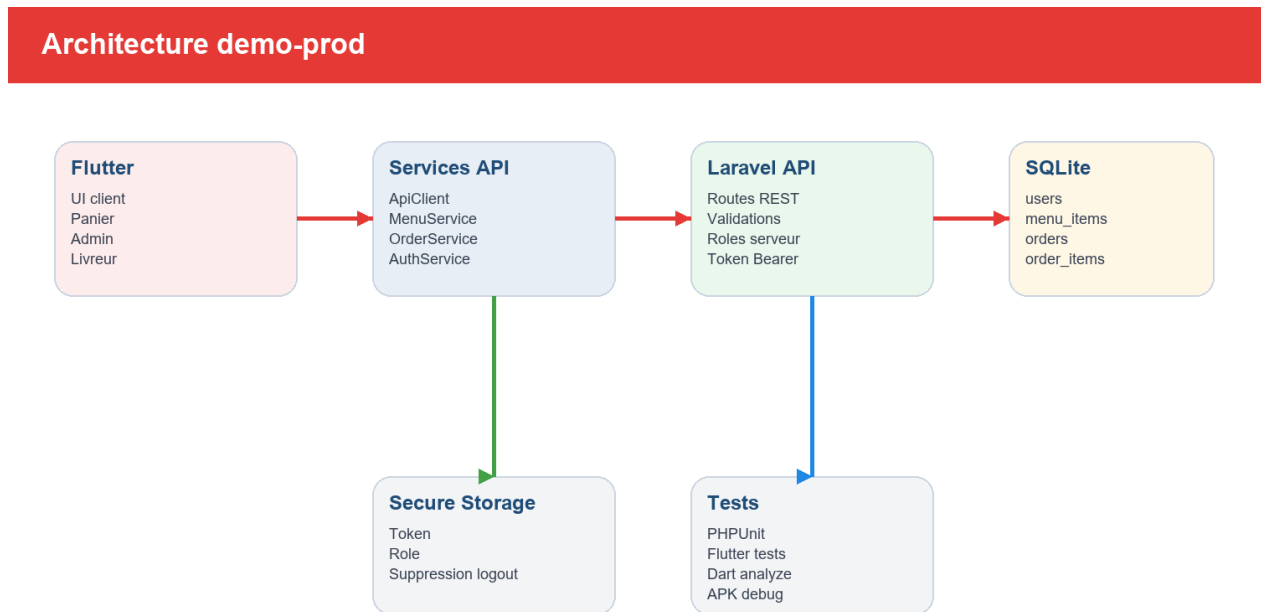


FIGURE 8.1 – Architecture démo-prod Flutter + Laravel

TABLE 8.1 – Choix techniques

Couche	Technologie	Justification
Frontend	Flutter / Dart	Un seul code pour Android, Windows et le web de test.
Backend	Laravel 13	Framework structuré avec routes REST, validations et tests.
Base de données	SQLite en démo	Démarrage rapide sans installer de serveur MySQL.
Sécurité locale	<code>flutter_secure_storage</code>	Conservation plus sûre du token côté mobile.
Tests	PHPUnit <code>flutter_test</code>	et Vérification automatique des flux principaux.

Le mode démonstration reste disponible pour garantir une maintenance fluide même si le serveur n'est pas accessible. Le mode backend réel s'active avec les variables `DEMO_MODE=false` et `API_BASE_URL`.

9 Réalisation backend Laravel

Le backend se trouve dans le dossier `mon_pfapi`. Il expose des endpoints REST publics et protégés.

9.1 Routes principales

TABLE 9.1 – Endpoints API

Méthode	Endpoint	Rôle
GET	/api/health	public
GET	/api/menu	public
POST	/api/register	public
POST	/api/login	public
POST	/api/logout	connecté
GET	/api/orders	connecté
POST	/api/orders	client
GET	/api/admin/stats	admin / serveur
GET	/api/driver/orders	livreur / admin

9.2 Composants backend

- `AuthController` : inscription, connexion, déconnexion et émission du token ;
- `MenuController` : catalogue public et endpoints de gestion menu ;
- `OrderController` : création commande, historique, statuts, file livreur et statistiques ;
- `TokenAuth` : middleware Bearer token et contrôle des rôles ;
- `DatabaseSeeder` : comptes de test, restaurant, catégories, menu et commande initiale.

10 Réalisation Flutter

L'application Flutter se trouve dans `mon_pfapp`. Elle contient les écrans pour l'authentification, l'accueil client, le menu, le panier, le suivi, le profil, l'administration et la livraison.

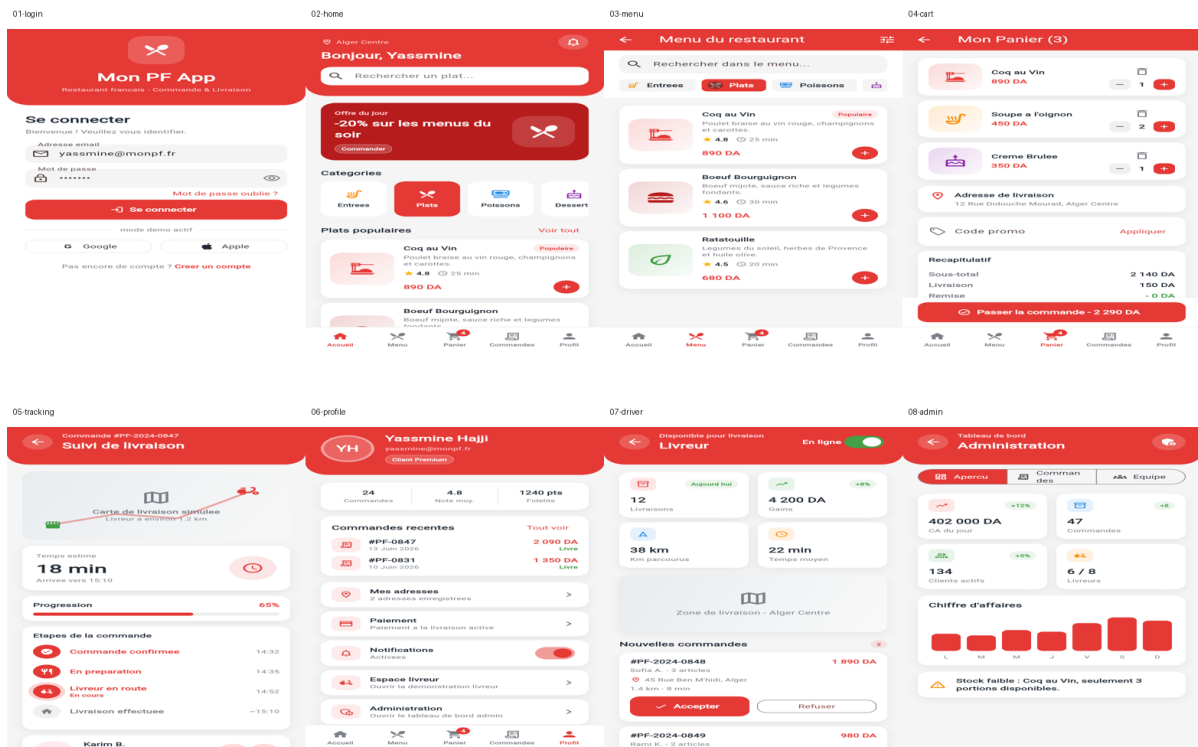


FIGURE 10.1 – Aperçu des écrans principaux de l'application

TABLE 10.1 – Fichiers Flutter importants

Fichier	Rôle
<code>lib/app/mon_pf_app.dart</code>	État global, navigation, panier, chargement menu et commandes.
<code>lib/data/api_client.dart</code>	Client HTTP avec headers, token, timeout et gestion d'erreurs.
<code>features/client/data/order_service.dart</code>	Création et récupération des commandes.
<code>features/auth/data/auth_service.dart</code>	Connexion, inscription et déconnexion.

`shared/widgets/app_ui.dart` Couleurs, cartes, navigation et visuels de plats.

L'interface utilise des images de plats, un logo personnalisé et des transitions animées afin d'obtenir un rendu proche d'un produit final.

11 Sécurité

La sécurité a été renforcée à plusieurs niveaux.

- la sélection de rôle a été retirée de l'inscription publique ;
- le backend force toujours le rôle `client` lors d'une inscription ;
- le token est stocké dans `flutter_secure_storage` côté application ;
- l'API ne stocke que le hash SHA-256 du token ;
- les appels API possèdent un timeout ;
- Android release refuse HTTP local et doit utiliser HTTPS.

TABLE 11.1 – Risques et réponses de sécurité

Risque	Solution
Auto-création admin	Rôle client forcé côté serveur
Token exposé localement	Secure storage côté app
Token brut en base	Stockage du hash uniquement
Serveur indisponible	Timeout et messages d'erreur
HTTP en release	Refus du cleartext sauf debug/profile

12 Algorithmes et règles métier

Le projet repose sur des algorithmes applicatifs simples et défendables.

12.1 Gestion du panier

Lorsqu'un utilisateur ajoute un plat au panier, l'application recherche si ce plat existe déjà dans la liste. S'il existe, la quantité augmente. Sinon, une nouvelle ligne `CartItem` est créée.

12.2 Calcul du total

Chaque ligne calcule son total par :

$$total_ligne = prix_unitaire \times quantite$$

Le total de la commande est ensuite calculé par :

$$total = sous_total + frais_livraison$$

12.3 Règles de commande

- une commande ne peut pas être créée sans plats ;
- les quantités doivent rester positives ;
- le statut est normalisé par le backend ;
- les rôles déterminent les actions autorisées.

13 Interface et expérience utilisateur

L'interface a été inspirée par la maquette restaurant fournie. Elle privilégie une présentation mobile claire : header rouge, cartes blanches, navigation basse, visuels de plats et boutons d'action visibles.

L'objectif UI/UX est de permettre au client de comprendre rapidement le parcours :

1. se connecter ;
2. consulter le menu ;
3. ajouter des plats ;
4. valider le panier ;
5. suivre la commande ;
6. consulter l'historique.

Les espaces admin et livreur utilisent des cartes de statistiques et des listes de commandes afin de rester lisibles sur téléphone.

14 Tests et validation

Les tests automatiques prouvent que l'application n'est pas seulement une maquette. Flutter teste les écrans critiques et Laravel teste les flux API.

TABLE 14.1 – Résultats de validation

Contrôle	Résultat
Dart analyze	No issues found
Flutter tests	5 tests passed
Laravel tests	5 tests passed
Flux API live	Login client et commande créée
Android debug APK	Build généré
Windows	Bloqué par composant ATL manquant

15 Installation et mise en service locale

15.1 Lancer le backend

```
cd "C:\Users\LOQ\Documents\yassmin pfe\mon_pfapi"
..\tools\runtime\php-8.4.22\php.exe artisan migrate:fresh --seed
..\tools\runtime\php-8.4.22\php.exe artisan serve --host=0.0.0.0 --port=8000
```

15.2 Lancer Flutter avec l'API locale

```
cd "C:\Users\LOQ\Documents\yassmin pfe\mon_pfapp"
C:\Users\LOQ\dev\flutter\bin\flutter.bat run --dart-define=DEMO_MODE=false --dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```

Pour un téléphone Android sur le même Wi-Fi, il faut remplacer 127.0.0.1 par l'adresse IPv4 Wi-Fi du PC.

16 Android et Windows

Android est la cible la plus avancée. L'APK debug existe et peut être installé avec ADB lorsque le téléphone est connecté et autorisé.

```
C:\Users\LOQ\AppData\Local\Android\Sdk\platform-tools\adb.exe
    devices
C:\Users\LOQ\AppData\Local\Android\Sdk\platform-tools\adb.exe
    install -r build\app\outputs\flutter-apk\app-debug.apk
```

Windows est une cible prioritaire, mais le build nécessite l'installation du composant Visual Studio ATL. L'erreur observée est :

```
fatal error C1083: Impossible d'ouvrir le fichier include : 'atlstr
.h'
```

La solution est d'ouvrir Visual Studio Installer et d'installer le composant **C++ ATL for latest v143/v14x build tools** ou l'équivalent disponible.

17 Difficultés rencontrées

TABLE 17.1 – Difficultés et solutions

Difficulté	Solution
Chocolatey bloqué par permissions	Installation portable PHP/Composer dans <code>tools/runtime</code> .
<code>dart.bat</code> lent ou bloqué	Utilisation de <code>dart.exe</code> direct du SDK Flutter.
HTTP local Android	Autorisation uniquement en debug/profile avec <code>tools:replace</code> .
ADB non trouvé dans PATH	Utilisation du chemin complet <code>platform-tools/adb.exe</code> .
Build Windows bloqué	Installation du composant Visual Studio ATL.

18 Validation du cahier de charge

TABLE 18.1 – Validation des exigences

Exigence	État
Client menu / panier / commande	Validé
Suivi de commande	Validé visuellement et par statut
Admin statistiques	Validé
Livreur file / acceptation	Validé
CRUD menu	Endpoints API disponibles ; UI admin à compléter
Synchronisation temps réel	Perspective d'amélioration
Déploiement HTTPS	Perspective d'amélioration

19 Perspectives

Les perspectives principales concernent la production réelle :

- héberger l'API sur un domaine HTTPS ;
- remplacer SQLite par MySQL ou PostgreSQL hébergé ;
- utiliser Laravel Sanctum pour une authentification plus standard ;
- ajouter des notifications push ;
- ajouter le paiement en ligne si le cahier de charge l'exige ;
- compléter l'interface admin de CRUD menu ;
- mettre en place une chaîne CI/CD GitHub Actions.

20 Conclusion

Mon PF App constitue une base solide de projet de fin d'études BTS. Le projet montre la capacité à comprendre un besoin, concevoir une architecture, développer une application, ajouter un backend, sécuriser les rôles et tester les flux.

La version démo-prod est suffisante pour une présentation claire : elle possède un parcours client, un espace admin, un espace livreur, une API Laravel, une base SQLite, un APK Android et une documentation détaillée.

Le travail restant est bien identifié et peut être présenté comme perspectives de production. Cette transparence renforce la crédibilité du projet.

A Comptes de démonstration

TABLE A.1 – Comptes seedés

Email	Rôle	Mot de passe
yasmine@monpf.fr	client	password
admin@monpf.fr	admin	password
serveur@monpf.fr	serveur	password
livreur@monpf.fr	livreur	password

B Fichiers importants

TABLE B.1 – Fichiers à connaître

Fichier	Rôle
<code>mon_pfapp/lib/app/mon_pf_app.dart</code>	État global et navigation
<code>mon_pfapp/lib/data/api_client.dart</code>	Client HTTP
<code>mon_pfapi/routes/api.php</code>	Routes REST
<code>mon_pfapi/database/seeder/DatabaseSeeder.php</code>	Données de démonstration
<code>docs/demo-prod-runbook.md</code>	Guide de lancement
<code>docs/backend-api.md</code>	Documentation backend